
Itérables

Exemple : Itérable

De nombreux objets sont itérables :

- les collections, les tableaux,
- les mots d'un texte,
- une séquence définie par une relation de récurrence
 $u_{n+1} = f(u_n)$,
- les pixels d'une image,
- ...

Besoin d'un mécanisme pour unifier le traitement.

Boucle `for` des itérables

```
Iterable<Pixel> image = ...;  
for (Pixel pix : image) {  
    doSomething(pix);  
}
```

- Syntaxe : `for` (EltType elt : provider) { ... },
- provider doit implémenter `Iterable<EltType>`,
- `break`, `continue` et `return` autorisés.

L'interface Iterable

```
public interface Iterable<E> {  
    Iterator<E> iterator();  
}
```

- Être itérable, c'est avoir la capacité de créer un objet chargé de l'itération (Iterator).

L'interface Iterator

```
public interface Iterator<E> {  
    boolean hasNext();  
    E next();  
    ...  
}
```

- hasNext permet de savoir s'il existe au moins un élément supplémentaire,
- next permet de récupérer le prochain élément (change à chaque appel).
- L'**itérateur** est l'objet responsable de fournir les éléments un par un.

Iterable vs Iterator

- Iterable **possède** des éléments qui peuvent être examinés un par un (peut être itéré).
- Iterator **fournit** des éléments un par un (itère).

Généralement :

- Suffixe **able** : indique une capacité.
- Suffixe **ator** : indique une responsabilité.

Itérateur d'entiers

- `cours3.Interval` représente un ensemble d'entiers consécutifs, est itérable ;
- `cours3.Range` génère des entiers consécutifs, est un itérateur.

```
Interval oneToTen = new Interval(1,10)
for (Integer i : oneToTen) { // valide
    System.out.println(i);
}
oneToTen.next(); // NON !
Range range = new Range(1,10);
System.out.println(range.next()); // valide
for (Integer i : range) { // NON !
    System.out.println(i);
}
```

Séquences récursives

Séquences définies par une équation de la forme $u_{n+1} = f(u_n)$,
 $u_0 = c$.

- `cours3.Sequence` représente la séquence, est itérable ;
- `cours4.SequenceIterator` génère les valeurs de la séquence, dans l'ordre, et est un itérateur.

Les collections

Pourquoi des collections d'objets ?

- pour manipuler des ensembles d'objets simultanément,
- pour ranger et accéder à des objets selon les besoins,

Les principales collections

Nom	Interface	Implémentations
Liste	List	ArrayList
Ensemble	Set	HashSet, TreeSet
File	Deque	ArrayDeque
Map	Map	HashMap, TreeMap

L'interface List

- Séquence indexée (depuis 0) d'éléments.
- Principales opérations :
 - `list.add(element)` ;
 - `list.get(index)` ;
 - `list.set(index, element)`.
- Itérable dans l'ordre des indices (boucle `for`).
- Alternative objet aux tableaux.

Consulter l'exemple `cours3.DictionaryWithList`.

L'interface Set

- Ensemble **non-ordonné** d'éléments.
- Principales opérations :
 - `set.add(element)` ;
 - `set.contains(element)`.
 - `set.remove(element)` ;
- Itérable dans un ordre arbitraire (boucle **for**).
- **Les éléments sont uniques** (pas de doublon).

Consulter l'exemple `cours3.DictionaryWithSet`.

L'interface Deque

- Séquence non-indicée d'éléments.
- Principales opérations :
 - `queue.offerFirst(element)`,
`queue.offerLast(element)` ;
 - `queue.removeFirst(element)`,
`queue.removeLast(element)`.
- Itérable dans l'ordre de la séquence (boucle `for`).
- Modifications aux extrémités uniquement (début First ou fin Last).
- Modélise les files (FIFO) et les piles (LIFO).

Consulter l'exemple `cours3.BreadthFirstSearchV5`.

L'interface Map

- **Table d'associations** (dictionnaire, fonction partielle) entre clés et valeurs.
- Principales opérations :
 - `map.put(key, value)` ;
 - `map.containsKey(key)` ;
 - `map.get(key)` (fournit la valeur associée) ;
 - `map.remove(key)` ;
- Les clés sont **uniques**, et forment un ensemble.
- Chaque clé est associée à une seule valeur.
- Itération sur les clés (avec `map.keySet()`) ou sur les valeurs (avec `map.values()`), ou les paires clé-valeur (avec `map.entrySet()`).

Consulter l'exemple `cours3.DictionaryWithMap`.

L'interface Collection

Set, Queue, Deque, List *étendent* l'interface Collection<E> (*mais pas Map*!).

Opérations principales :

- `boolean isEmpty()` ;
- `int size()` ;
- `Iterator<E> iterator()` qui permet d'itérer avec une boucle `for`.

Itérer sur une collection

Comment faire une manipulation pour chaque item d'une collection ?

La boucle **for** permet de faire les mêmes instructions pour tous les éléments d'une collection.

```
Collection<Item> itemList = ...;  
for (Item item : itemList) {  
    ... // instructions utilisant item  
}
```

Les éléments sont traités un par un. **Ordre des éléments** : dépend de l'implémentation de la collection.

Trier une liste

La classe Collections (notez le 's') contient des fonctions (méthodes statiques) pour travailler sur les collections.

Exemple :

```
List<Item> myList = ...;  
Collections.sort(myList);
```

Pour trier, il faut que la classe Item définisse une méthode compareTo (qu'elle implémente l'interface Comparable<Item>), ce qui permet d'ordonner les Item.